MEDICINE REMINDER

MERN Stack Web Application

PROJECT REPORT

Submitted in partial fulfillment of the academic project requirements

Student Name: Umashankar Kongari

Project: Medicine Reminder

Technology: MongoDB, Express.js, React.js, Node.js

Academic Year: 2026–2027

ABSTRACT

The Medicine Reminder is a web-based healthcare management application designed to help users organize medicine schedules. The system manages medicine name, dosage, reminder time, start date and end date. The project follows the MERN stack architecture, using React.js for the user interface, Node.js and Express.js for REST API development, and MongoDB for data storage. CRUD operations allow relevant medicine records to be added, viewed, updated and managed. Authentication and role-based access can be incorporated where required. The application provides a centralized and user-friendly way to maintain medication schedules and demonstrates full-stack web development concepts.

1. INTRODUCTION

Forgetting medicine schedules can make it difficult for users to follow a planned medication routine. A digital reminder system can organize medicine details and scheduled timings in one place. This project develops a MERN-based Medicine Reminder website that allows users to manage medication records through a web interface.

2. PROBLEM STATEMENT

Build a MERN-based medicine reminder system that manages medicine schedules. The application should store medicine details such as medicine name, dosage, time, start date and end date, and provide the required operations for managing these records.

3. DETAILED DESCRIPTION

The system uses React.js to provide the frontend user interface. Node.js and Express.js handle backend processing and REST APIs, while MongoDB stores medicine-related records. The application is designed around the basic operations required for a medicine management system: adding new records, viewing existing records, updating information and managing/removing records where appropriate.

4. OBJECTIVES

- Demonstrate MERN Stack development.
- Implement CRUD operations using MongoDB.
- Build REST APIs using Node.js and Express.js.
- Develop the frontend using React.js.
- Implement authentication and role-based access where required.
- Display project information and results clearly.

5. SCOPE OF THE PROJECT

The project focuses on digital medicine schedule management. A user can maintain medicine information and scheduled dates/times through the web application. The architecture can be extended later with notifications, adherence history, doctor/caretaker access and mobile support.

6. TECHNOLOGY STACK

React.js: Builds the interactive frontend and reusable UI components.
Node.js: Provides the server-side JavaScript runtime.
Express.js: Provides backend routing and REST API functionality.
MongoDB: Stores medicine and user records in a document-oriented database.
HTML/CSS/JavaScript: Support the web interface and presentation.
Git/GitHub: Used for source-code version control and project hosting.

7. SYSTEM REQUIREMENTS

Hardware: Computer/laptop, minimum 4 GB RAM recommended, and internet connection for development/deployment activities.
Software: Modern web browser, Node.js, npm, code editor such as Visual Studio Code, MongoDB/MongoDB service, and Git.

8. SYSTEM ARCHITECTURE

The application follows a three-layer full-stack architecture. The React frontend sends HTTP requests to the Express/Node.js backend. The backend validates and processes requests and communicates with MongoDB. The resulting data is returned to React and displayed to the user.

Flow: User → React Frontend → Express/Node.js REST API → MongoDB → API Response → React UI.

9. MAIN MODULES

User Authentication: Provides login/registration functionality when authentication is implemented.
Medicine Management: Stores medicine name, dosage, time and date range.
Reminder/Schedule Management: Organizes the planned time and validity period of medicines.
CRUD Operations: Allows records to be created, viewed, updated and managed/deleted.
Dashboard: Presents project information and relevant medicine records clearly.

10. DATABASE DESIGN

A MongoDB medicine document can contain fields such as medicineName, dosage, reminderTime, startDate and endDate. User-related information can be maintained separately when authentication is implemented. MongoDB's document model is suitable for storing structured medicine records and extending them with additional fields in the future.

11. CRUD OPERATIONS

Create: Add a new medicine schedule.
Read: Display stored medicine records.
Update: Modify dosage, time or date information.
Delete/Manage: Remove or manage a medicine record when it is no longer required.

12. REST API DESIGN

The backend can expose REST endpoints for medicine management. Typical operations include GET for retrieving records, POST for adding a medicine, PUT/PATCH for updating a medicine, and DELETE for removing a medicine. API responses should use appropriate HTTP status codes and JSON data.

13. WORKING OF THE SYSTEM

First, the user opens the Medicine Reminder website. The frontend displays the available medicine-management interface. The user enters medicine details and submits the form. React sends the data to the backend API. Express processes the request and stores the record in MongoDB. Stored records can then be retrieved and displayed in the frontend. Existing records can be edited or removed using the corresponding management operations.

14. TESTING

Testing verifies that the major functions behave as expected. Functional testing should cover form validation, adding records, displaying records, editing records, deleting records, API responses and database connectivity. Authentication features should also be tested if enabled.

15. SAMPLE TEST CASES

The following test cases can be used for the project demonstration.

Test Case	Input/Action	Expected Result
Add medicine	Enter valid medicine details	Record is added
View medicines	Open medicine list	Saved records displayed
Update medicine	Change dosage/time	Record is updated
Delete medicine	Remove selected record	Record is removed
Invalid data	Submit incomplete/invalid form	Validation message shown
API request	Send valid CRUD request	Correct JSON/status returned

16. ADVANTAGES

- Centralized medicine schedule management.
- Simple web-based interface.
- Demonstrates complete MERN-stack development.
- Supports database-backed CRUD operations.
- Can be extended with notification and monitoring features.

17. LIMITATIONS

The current project is primarily focused on medicine schedule management. Reminder delivery may depend on the implemented notification mechanism and browser/device permissions. The system should not be treated as a substitute for professional medical advice.

18. FUTURE ENHANCEMENTS

Possible future improvements include browser/mobile notifications, SMS or email reminders, medicine history and adherence reports, prescription upload, caretaker accounts, doctor integration, recurring schedules, accessibility improvements and a dedicated mobile application.

19. CONCLUSION

The Medicine Reminder project demonstrates how the MERN stack can be used to develop a practical full-stack web application. React provides the user interface, Node.js and Express.js provide the backend API layer, and MongoDB provides persistent data storage. The project meets the core objective of managing medicine schedules while providing a foundation for future healthcare-oriented features.

20. REFERENCES

1. Project source repository: GitHub – Umashankar2951/Medicine-Reminder.
2. MongoDB documentation.
3. React.js documentation.
4. Node.js documentation.
5. Express.js documentation.